

Comparative Analysis

Name: Anto Melwin S

Rollno: 192511299

Course Code: CSA0311

Course Name: Data Structures

CO4-AT1

1]

Compare Linear Probing, Quadratic Probing and Double Hashing.

In hashing, when two keys produce the same hash index, a collision occurs. Open addressing methods resolve collisions by finding another empty location in the hash table.

Linear probing. It checks the next consecutive positions until an empty location is found.

$$h_i(k) = (h(k) + i) \bmod m.$$

where $i = 0, 1, 2, \dots$ and m is the table size.

Advantages:

Very simple to implement

Good cache performance because locations are accessed sequentially.

Requires no additional memory.

Disadvantage:

Causes primary clustering, where consecutive occupied position form large blocks.

As the load factor increases, the number of probes increases significantly.

Quadratic Probing: It uses a quadratic function to determine the next position.

$$h_i(k) = (h(k) + c_1 i + c_2 i^2) \text{ mod } m.$$

Instead of checking consecutive locations, it checks positions at increasing quadratic distances.

Advantage: Reduces primary clustering compared with linear probing.

Provides better distribution of keys.

Disadvantages: Can still suffer from secondary clustering, where keys having the same initial hash value follow the same probe sequence.

Proper table size and constants are required for good performance.

Double Hashing: It uses two independent hash functions.

$$h_i(k) = (h_1(k) + i \times h_2(k)) \text{ mod } m.$$

Advantages: Minimizes both primary and secondary clustering.

Provides better distribution of keys.

Usually requires fewer probes than linear and quadratic probing.

Disadvantage:

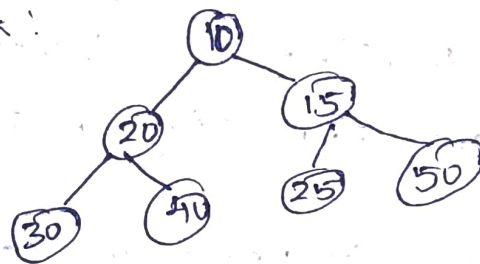
More computation is required because two hash functions are used.

2] BinaryHeap vs. Balanced Binary Search Tree

A scheduler needs to manage tasks according to their priority. Suppose the task with the smallest priority value must be executed first. A min-heap or a balanced BST such as an AVL tree or Red-black tree can be used.

Binary min-Heap: A binary min-heap is complete binary tree in which every parent has a value smaller than or equal to its children.

ex:



Operations:

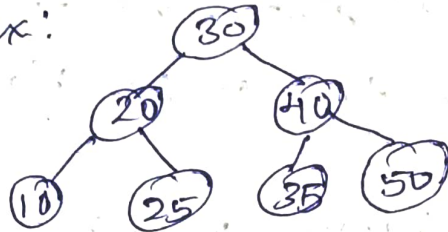
Insert: A new task is inserted at the end and moved upward using heapify-up.
 $O(\log n)$.

Extract-min: The root is removed, the last element is placed at the root and heapify-down is performed. $O(\log n)$.

Decrease Key: When a task's priority becomes higher, its key can be decreased and moved upward. $O(\log n)$.

Balanced Binary Search Tree: An AVL tree or Red Black tree. Its height is approximately: $O(\log n)$. This allows efficient searching and updating.

ex:



Operations:

Insert: A new key is inserted and the tree is rebalanced if necessary. $O(\log n)$.

Extract Min - The leftmost node contains the minimum value. Finding and removing it takes $O(\log n)$.

Decrease Key: The key can be updated while maintaining the BST properties. $O(\log n)$ which is more memory-efficient.

The binary heap is more memory-efficient because it can be represented using a simple array: $[10, 20, 30, 15, 40, 25, 50]$

It does not require separate left child, right child and parent pointers for every node.

A balanced BST generally requires more objects and pointers, which creates additional memory overhead.